

StockAgent

Technical design & knowledge transfer

Understand the three loops, IPO evidence, portfolio decisions, profit and loss, scheduled work, and the changes still ahead.

Edition 2026-09-29

Code-grounded explanations for engineering and product teammates.

Human testing duties are a separate document.

Source: docs/TECHNICAL_DESIGN.md · SHA-256 70a321ef44a4

Inside this guide

1. Reading the evidence

2. Product map and three loops

3. Runtime, configuration and storage

4. Research: input to recommendation

5. Learning: forecast, review and memory

6. Portfolio: advice and virtual execution

7. Profit/loss and the different marksheets

8. Discovery and IPO functionality

9. Scheduled jobs and event hooks

10. Delivery, interface, identity and operations

11. Changes already present and planned redesign

12. PI changes included now

13. Validation and KT sequence

Edition: 2026-09-29 · **Audience:** engineers and teammates learning the product

Code inspected: 165d152aa5ac52419cafc0ea6a4d68a7c031f270

First edition 2026-09-15 at 9a805878; maintained per story since. The revision above is the one the whole body describes. `check_kt_docs.py` fails if a linked source file or a documented job ID is absent at that revision.

PDF: StockAgent-Three-Loops.pdf, generated from this Markdown.

This is the current KT source, replacing the May technical reference. It explains the implementation and the changes planned in PI-2026-09. The separate human testing guide divides practical testing duties; it is not an implemented human-approval workflow.

1. Reading the evidence

Label	Meaning in this edition
Current code	Traced in this checkout. Flags, inputs and runtime data determine whether a path actually runs.
Locally checked	Existing tests run in an isolated copy. Exact results and limits are in the validation receipt.
Production observation	Dated evidence: the September 10 audit, section 10's 2026-09-21 email diagnosis, a September 15 deployment SUCCESS at 9a805878, the 2026-09-23 read-only log inspection of learned weights, and the 2026-09-24 SA-039 weight baseline. Those logs came from deploy d9c459ae (commit e8df088). Deploys carrying the header revision's code were inspected read-only on 2026-09-25 (7ebd06c5: the 16:30 review ran in adapt) and on 2026-09-26 (da9df6cf: all 20 tickers in observe, per the activation record).
PI target	Intended behavior, not completed functionality. SA-039 was accepted by its fresh review on 2026-09-25, and production has run observe since 2026-09-26 (activation record; verification after the next reviews is pending). SA-001 (chat rendering) was accepted by its fresh review on 2026-09-26 (review); it is committed as 8413b59, and its production verification is pending. SA-002 (data health) was accepted by its fresh re-review on 2026-09-26, after a rework of its test fixtures and one fundamentals case (receipt, review); it is committed as 8413b59, and its production verification is pending. SA-003 (the decision gate) was accepted by its fresh review on 2026-09-27 (receipt, review); it is committed as 167f08b, and its production verification is pending. It ships recording only (decision_gate.mode: record), and enforcing it is a separate decision, after SA-039's observation window and a measured record period. SA-004 (daily-review outcome counts) was accepted by its fresh review on 2026-09-27 (receipt, review); it is committed as 241c393, and its production verification is pending. SA-005 (a hermetic test suite and a CI workflow) was accepted by its fresh review on 2026-09-28 (receipt, review); it is committed as 48143ed, and its CI workflow has not run yet. SA-006 (delivery transport and dead letters) was accepted by its fresh re-review on 2026-09-28, after a rework for two review findings (receipt, review); it is committed as 15dcda1 and not yet deployed. SA-007 (independently recoverable backups) was accepted by its fresh review on 2026-09-29, with two low follow-ups routed to SA-034 and SA-036 (receipt, review); it is committed as 165d152 and was deployed on 2026-09-29 (26b442f4). No off-site bucket exists yet (the owner deferred it to the end of the PI). Change 1, which makes the missing-bucket reminder weekly instead of daily, was accepted by its fresh review on 2026-09-29 and is not yet committed. Every other story from SA-008 to SA-051 is todo. Three are stretch.

The September audit records unresolved label, timing, health, weight-bound and operational defects. Adaptive state exists; improved prediction performance has not been demonstrated. No production job, backfill or notification was triggered for this KT.

Vocabulary

Term	Plain-English meaning
Ticker / instrument	A market symbol / the actual security it represents. A renamed or reorganized company is not automatically the same investment.
Sector / graph sector	The company's business sector / the analysis implementation selected for it. A pharma stock can use a generic graph while retaining pharma identity.
LLM / agent / dimension	Language model / component with an analysis task / one score such as fundamentals. Multiple dimension names do not establish independent evidence.
Composite / weight	Combined score / importance assigned to a dimension.
Envelope	Stored forecasts for future trading sessions, including prices, verdicts and uncertainty information.
Feedback / lesson / dossier	One review result / reusable knowledge / longer-lived company observations and questions.
Advice / transaction	A portfolio recommendation / a recorded virtual buy or sell. They are separate records.
Cohort / matured outcome	A defined group of predictions / an outcome whose observation period has finished.
Idempotent	Repeating the same operation does not apply its effect twice.
Cache / projection	A reusable fetched result / a secondary representation of another store. Neither guarantees freshness or authority.
PI	Improvement plan; accepted story statuses are separate from features already present in the baseline.

2. Product map and three loops

StockAgent combines listed-equity research, adaptive feedback, a virtual portfolio, discovery, IPO information and reports. Three connected product loops explain the main flow:

1. **Research:** collect evidence, score dimensions, combine them and produce a report.
2. **Learning:** compare forecasts with observations, record feedback, adjust weights and maintain lessons and company knowledge.
3. **Portfolio:** evaluate held positions, record advice, optionally execute it in a virtual account, then produce transactions, value history and summaries.

The August **Three Loops PI** named an improvement program covering routing, data health and grading. Its proposed redesigns are not all implemented.



The nightly outcome auditor measures issued advice. The watchdog checks operational milestones. Neither is the daily feedback agent. There is no general human-approval queue between these components.

Repository orientation

Location	Responsibility
API server and routes	FastAPI startup, API and streaming interfaces.
src/backend/shared	Shared orchestration, canonical schemas, prompts and settings.
src/backend/sectors	Native implementations, generic graph and sector registry.
services/data	Fetchers, context assembly, caches, durable stores and backup.
core/intelligence	Forecasting, feedback, regimes, seasonal logic and company knowledge.
core/portfolio, core/discovery, core/ipo	Virtual money path, listed-stock discovery and IPO evidence.
core/audit, core/ops/watchdog, core/delivery	Outcome measurement, operational monitoring and delivery.
src/frontend/prototypes	Served React/JSX application, PWA and data adapters.
tests	Unit, integration and contract checks; their presence alone is not acceptance.

Some paths are compatibility re-exports. For example, `core/config/settings/base.py` forwards to `src/backend/shared/config/settings/base.py`. Follow the import before editing; two import paths need not mean two implementations.

3. Runtime, configuration and storage

Runtime

Dockerfile uses Python 3.11 and runs `services.api.server:app` with two Uvicorn workers. It copies `src/backend/` to `/app/backend/`; local tests add `src` to the import path. The served frontend is the prototype JSX application. `package.json` provides development tooling; there is no compiled app frontend build step. The checked-in C# scheduler is not launched by this Dockerfile.

Both workers run `startup`. One binds a localhost TCP port, default 59321, and becomes the background owner. It starts RL self-heal, APScheduler, cleanup and the optional Atlas outbox drainer; the other serves API requests. This guard is container-local, not a distributed lock or continuous standby takeover. `GET /health` returns process-level ok; it does not verify background ownership, storage, jobs, data or delivery. SA-029 addresses that gap.

Configuration

The loader resolves a declared environment override, then `config.yaml`, then a code fallback. Not every YAML field has an environment override. Most settings are resolved at import, so editing a file does not imply live reconfiguration.

Checked-in setting	Meaning
<code>scheduler.enabled: false</code>	Local startup default; production overrides are separate.
<code>scheduler.feedback_cron: "30 16 * * mon-fri"</code>	Daily review default, interpreted in Asia/Kolkata.
<code>rl.hard_bind_verdict_enabled: true</code>	Research category follows the composite; model <code>final_score</code> remains separate.
<code>rl.learning_mode: adapt</code>	SA-039 switch, env <code>RL_LEARNING_MODE</code> . <code>adapt</code> keeps learned weights and lesson emphasis live; <code>observe</code> contains them (section 5). Any other value fails closed to <code>observe</code> .
<code>decision_gate.mode: record</code>	SA-003 switch, env <code>DECISION_GATE_MODE</code> . <code>record</code> computes and records the essential-data gate everywhere and changes nothing; <code>enforce</code> withholds verdicts, forecasts, learning and new-risk buys that rest on unverified data (sections 4–6). Any other value fails closed to <code>enforce</code> .
<code>rl.control_lane_enabled, rl.scorecard_enabled: true</code>	Control/evaluation paths configured; not proof of valid prospective comparison.
<code>ipo.enabled: true; ipo.gmp_enabled: false</code>	IPO refresh and grey-market-price fetching have separate gates.
<code>delivery.enabled: true; delivery.email_enabled: false; delivery.push_enabled: true</code>	Job scheduling, transport enablement and credentials are separate conditions.
<code>watchdog.enabled: true; watchdog.prep_enabled: true</code>	Monitoring can perform configured prep as well as report results.
<code>audit.enabled: true</code>	Nightly outcome grading configured.

These are repository values, not a fresh inventory of production variables. Credentials and .env contents do not belong in KT or testing evidence.

Persistent records

Records	Writer / consumers and scope
data/predictions/<sector>/<ticker>/	PredictionStore: envelopes, feedback, controls, weight memory, lessons and dossier, plus <TICKER>_weight_observations.json (SA-039 observe-mode proposals). RL writes; advisor, evaluation and UI read. SA-009/SA-017 still need to reconcile historical aliases and mixed stores.
_shared_ledger.json per sector; _market_ledger.json at prediction root	Shared lessons can affect more than one ticker; bad attribution can propagate.
data/portfolio/<user>/	PortfolioStore: holdings/cash, advice and transaction JSONL, value history, dated digests, briefs and weekly reviews.
data/ipo/	Historical issue facts and captured demand snapshots; reports derive from these records.
data/market_cache/ipo.json and EOD parquet	Cached IPO information and exchange price history; date, source and stale state matter.
SQLite score, log, user and Atlas stores	Historical scores, telemetry, sessions and relational indexes/outbox. Secondary tables may lag primary records.
data/scheduler_job_outcomes.json; data/watchdog_state.json	Operational results and milestone state, not proof of financial performance.
data/backups/	Local rotation of the last 7 archives, each with its manifest, and backup_status.json (the last restore drill and off-site result). SA-007 adds an encrypted copy outside the volume once the owner configures one; see section 10. A local archive is not off-site recovery.

PortfolioStore's JSONL ledger is the inspected advice source. September 10 found incomplete/stale optional Atlas projections. An empty projection does not prove no advice was issued.

4. Research: input to recommendation

Trace: registry → base orchestrator → bundle builder → unified analyst → aggregator.

1. Resolve input to a company, symbol and sector. Known managed symbols can bypass model-based resolution.
2. Select the graph. Automobile, banking/BFSI, IT and renewable energy have native implementations; other sectors use generic analysis. The RL router delegates graph selection to the central registry; August A1 is present.
3. Assemble company/sector news, fundamentals, technicals, macro, flows, peers and company knowledge. NSE prefetch data is shared. Sections may be cached, unavailable or not applicable.
4. The preferred unified analyst scores the sector dimensions together. The legacy per-dimension pool remains a fallback. A whole research request can involve resolution, aggregation, retries and extra provider calls; it is not necessarily one LLM call.

5. The aggregator combines weighted scores, excludes errored contributions and normalizes surviving weights. With no surviving weight it falls back to a neutral composite. An LLM produces narrative and report fields.
6. With hard-bind enabled, configured score bands determine the categorical verdict. The raw model verdict is logged for comparison; its numeric `final_score` remains separate.
7. Score history, run summaries, logs and data-health records support the user report and later diagnosis.
8. The decision gate (SA-003, below) labels the report actionable, degraded or abstain before anything logs or returns its verdict.

Example: scores 0.8 and 0.4 at weights 0.75 and 0.25 give composite $0.8 \times 0.75 + 0.4 \times 0.25 = 0.70$. The configured band containing 0.70 gives the bound verdict. A different LLM `final_score` is a different field, not necessarily an arithmetic error.

Data health (SA-002: accepted 2026-09-26 by its fresh re-review, after a rework; committed as 8413b59; production verification pending). Every unified run writes one data-health row, to `data/logs/data_health.jsonl` and `telemetry.db`. Before SA-002 a section's status was guessed from its text, so any nonempty sentence read ok, including "Technical data unavailable for TATAMOTORS" and "[No results for: ...]". The row's health looked at dimension counts only. On 2026-08-26 two TATAMOTORS runs against a price source answering HTTP 404 were recorded ok, with 10 live sections and 9/9 dimensions. Now every section producer returns a typed result (`fetch_result.py`): a status, the source, the as-of date of its newest datum, and a reason. The producer decides the status from its structured data, not from its sentence.

- ok and `cache_hit` mean verified data. `stale` means older than the freshness bound: a newest price bar over 7 days old, or a newest reported quarter over 200 days old. `fallback` means a core figure was replaced by search snippets or substituted zeros, or the newest listed quarter has no figure (yfinance lists an announced quarter as NaN before its results are filled in; `as_of` is then the newest quarter with figures). An older quarter without a figure is named in the reason only. `empty` means nothing came back; `n/a` means deliberately not fetched; `failed:<Type>` means an exception. `unverified` means plain text with no typed result.
- A row is ok only when every dimension scored and every applicable section is verified and fresh. Otherwise it is degraded, or hollow (thresholds unchanged), and `health_reasons` says why. Fundamentals, technicals and peers valuation are also listed in `essential_unusable`; SA-003's gate reads them.
- A Tavily month-cache entry now records how many results it held, so a cached "no results" stays empty for the rest of the month.
- New rows carry `contract_version`: 2. Older rows stay as written and are read as version 1 with unknown provenance, not upgraded.

Example: the price source answers 404 for TATAMOTORS while news, macro and flows answer. The analyst's prompt text is unchanged. The row now says degraded with `essential technicals=empty`, `essential fundamentals=empty` and `essential peers_valuation=fallback`, where it used to say ok. The decision gate below reads the same statuses.

Decision gate (SA-003: accepted 2026-09-27, committed as 167f08b, production verification pending; ships recording only). Before SA-003 the row was write-only, and production recorded BUY on 1 of 6 scored dimensions and STRONG BUY on 3 of 9: the aggregator renormalises over whatever was scored, so one surviving dimension becomes the whole verdict. Every report now carries a typed `decision_gate` (module `decision_gate.py`), computed from the same section statuses and dimension counts as the health row, whether or not the row is recorded:

- **abstain** when an essential section (`observability.data_health_essential_sections`) is not `ok/cache_hit`, when fewer than half the sector's dimensions were scored (`decision_gate.min_dimensions_fraction: 0.5`), or when the run has no bundle at all (the legacy worker-pool fallback has no provenance). An unresolved symbol lands here too: no provider knows it, so its essential sections are empty.
- **degraded** when it may act but an ordinary enrichment (news, macro, flows, a dimension above the floor) is missing. Degraded is actionable.
- **actionable** when nothing is missing.

One switch, `decision_gate.mode` (env `DECISION_GATE_MODE`). `record`, the checked-in value, changes nothing: every consumer computes the gate and records what enforcement would stop, in `data/logs/decision_gate.jsonl`, on the report, on forecast rows and on advice. In `enforce` an abstaining report's verdict reads `INSUFFICIENT DATA` (the aggregator's verdict is kept as `withheld_verdict`), and sections 5 and 6 describe what the learning and portfolio consumers then refuse. An unrecognised value fails closed to `enforce`. **Example:** the TATAMOTORS 404 run above scores 9 of 9 and the aggregator says **STRONG BUY**. Its gate is `abstain`, naming the three essential sections. In `record` the user still sees **STRONG BUY** and the gate log gains a row saying what enforcement would have withheld; in `enforce` the user sees `INSUFFICIENT DATA`. The same run with its price source answering stays `actionable` and keeps **STRONG BUY**.

Current gap: production records the gate only once SA-003 is deployed, and nothing is withheld until the owner switches to `enforce`. How often the gate would abstain is not yet measured: a blank newest quarter, for example, makes fundamentals `fallback` and so abstains, for weeks after each quarter end. SA-008 handles instrument lifecycles (renames, demergers, successors); SA-010 corrects benchmark arguments. SA-025 measures calls before SA-026 consolidates sector definitions and SA-027 retires only justified fallback duplication.

Planned redesign (adopted 2026-09-26, not implemented). The one-engine design retires the per-sector graphs. Today all five graphs already run one bundle and one reasoning-model call; the LangGraph pool is only a fallback. They differ mainly in five scoring schemes (37 dimension slots under 24 names), and the LLM re-scores numbers the code has already computed. Generic-graph stocks are valued against automobile peers, and missing data is scored 0.5. The plan has four parts: - sector knowledge becomes a lens YAML (peers, benchmark, KPIs, news terms and macro drivers), resolved from NSE's industry field; - code computes five factors (Value, Quality, Growth, Momentum and Risk) as percentiles against real peers and the stock's own history; - one LLM reader returns dated, sourced events, and code turns them into a sixth factor, Catalyst; - code combines the six at equal weights and maps the result to the existing bands. With fewer than 4 of 6 factors it gives no directional verdict. The LLM explains the verdict but cannot change it.

SA-044 fixes the peers first. SA-026, SA-045 and SA-046 run in shadow. SA-047 switches only when the engine is right on at least 50% of disagreements with today's analyst, and SA-027 then deletes the sector packages.

5. Learning: forecast, review and memory

Trace: forecast generation, interpolator, daily review, feedback agent, weight adapter, prediction store, learning mode.

Forecast generation

The workflow loads weights and knowledge, runs analysis and creates an envelope across future exchange sessions. A model-shaped profile, static fallback and simulated paths supply forecast prices and bands. Generated forecast rows are not independent observed samples or guaranteed probabilities.

The monthly trigger is the **calendar first at 09:00 IST**. The generated rows use exchange trading dates. Do not call that trigger the first trading day. Shocks, thesis changes and later evidence can revise/archive envelopes. SA-013 will define immutable issuance identity so revised knowledge does not replace the earlier decision's evidence.

SA-003: every forecast row now names the analysis run that issued it (`source_run_id`) and that run's gate (`data_gate`); the envelope keeps the latest run's full `decision_gate`. A row issued before SA-003 has an empty `data_gate`, meaning provenance unknown. In `enforce` mode an abstained analysis builds no envelope (`InsufficientDataError`; the monthly job logs it) and regenerates nothing on a shock (the current envelope stays as it is). Nothing scheduled retries a withheld month-start envelope before the next month's run: only a restart's self-heal regenerates a missing envelope. Until then that ticker's reviews return `no_envelope` and its ADDs stay blocked. A same-month retry is routed to SA-036.

Daily review walkthrough

1. The scheduler selects the previous exchange trading session.
2. Daily review finds its envelope row and obtains the actual close, including the configured verification path.
3. It computes price error and the current direction label. Some small, correct misses skip a fresh analyst run.
4. Otherwise it may rerun analysis. With `hard-bind` enabled, current code can grade the old outcome against the fresh report's verdict.
5. Feedback classifies misses and attributes them to dimensions; weights, lessons and other learning state are updated.
6. Conditional thesis review, reforecasting, dossier curation and shared knowledge updates can run under their feature gates.
7. After harvesting reviews, the scheduler invokes the portfolio pipeline. Returning successfully does not establish that every ticker got feedback.

Decision gate in the review (SA-003). The review checks three inputs, in order, before it writes anything: - the graded row came from an actionable analysis (not `abstain`, not `unknown`); - the actual close is from the bar dated the review session. The close fetchers fall back to the newest earlier bar when the session has none (a suspended symbol, a provider lag), and that fallback is unchanged. What is new is that the result names the bar it came from. A known case (routed to SA-012): when NSE has not yet listed the session and `yfinance`'s session close differs from NSE's latest row by more than 1%, the cross-check still prefers NSE's earlier close. That close is now marked `stale`, not `graded`; - the fresh re-run, when one happens, is `actionable`.

In `enforce` mode the first failure ends the review as `data_gated`: no grading, `FeedbackAgent` call, weight update or `observe-mode` proposal, lesson, envelope revision, streak, feedback entry, dossier or control-lane step. The skip is recorded with its stage, reason and source run id. The market-wide sticky regime, updated at the start of every review, is the one write that still happens. In `record` mode the review runs as before and its summary lists what enforcement would have stopped. **Example:** a symbol is suspended on the review day, and both providers return the previous day's close of 99.0. That used to be graded as a flat day, a zero return that never happened. Now the close says it is from the previous day's bar, and enforcement skips the review. Since SA-004 (accepted 2026-09-27, committed as 241c393) the scheduler counts a `data_gated` review on its own: it is not output, so it cannot quiet the zero- or partial-output alert, and a completed review that lists `data_gate` inputs counts as `degraded` (section 9).

Grading defect: an example everyone can follow

Suppose the issue-time close was 100, the predicted close 110 and the observed close 105. The stock rose **5%**, but undershot its forecast by **4.55%**. Current `classify_direction(actual, predicted)` compares against the forecast and labels this `DOWN`. That measures error direction, not investment direction. A new verdict computed

after observing the outcome can also become the graded verdict. Label definition and issue-time information are separate problems.

Existing tests can describe those code paths without establishing the desired financial contract. SA-012 defines the target; SA-013/SA-014 freeze and grade information available when the decision was issued.

What adaptation means here

Weights, accuracy history, lessons and dossier records change over time. Regime and lesson modifiers affect research inputs. This is an adaptive feedback system; it does not demonstrate that adaptation beats a fixed policy. The audit also reproduced final weight-bound violations and repeated ensemble trend counts presented under different agent names.

Production adapts live weights on every eligible review. On 2026-09-23, production logged `technical` at 0.0 for 5 of the 6 tickers that have that agent. The 2026-09-24 baseline reproduced it: the defaults are 0.12 in the generic graph and 0.10 in the renewable graph. Counting `pattern_analysis`, the chart agent of the other graphs, 9 of 18 tickers had a chart weight of 0.0 and 16 of 18 were below half their default. All 19 reviews that day wrote a new weight version.

SA-039 (accepted 2026-09-25; production observe since 2026-09-26): `rl.learning_mode` has two values. `adapt` is the checked-in value and the behaviour described above. `observe` contains learning:

- Forecasts, re-forecasts, public analysis, and the review's re-scoring and forecast revision aggregate with the sector's configured default table (the table a ticker with no learning state gets), whatever the file stores.
- The review never writes weight memory. The adapter runs on a copy, and its would-be weights, deltas and reason go to `<TICKER>_weight_observations.json`, one record per review date.
- Tagged-lesson emphasis and the older category micro-adjustment stop moving agent scores. Lessons are still recorded, and `claims_fired` stays empty because no claim acted.

Example: a ticker stores `technical = 0.0` at v41. In `observe` mode its forecast uses 0.12; after the review the file still says 0.0 at v41, and the observation record shows what v42 would have been. Switching back to `adapt` resumes the stored weights unchanged. An unrecognised value fails closed to `observe` with a warning. The mode is shown in the review's start and completion log lines, on each envelope (`learning_mode`), and by `/ui/rl/weights/{ticker}` (with the live `decision_weights`) and `/ui/rl/summary/{ticker}`. The paper lane still never trains or writes weights, and the absurd-price-error guard still skips the adapter. Like every other decision path, the paper lane uses the sector defaults in `observe`. SA-039 does not contain regime multipliers, thesis multipliers, the conviction streak, miss-counter prompt enhancements or dossier text. **Production has run observe since 2026-09-26 at about 06:40 IST.** The owner set the Railway variable `RL_LEARNING_MODE=observe`, which overrides the checked-in `adapt` (activation record). Rollback is deleting the variable. Forecast rows issued before activation keep their verdicts and closes; the review re-weights only their confidence. The first fully contained cycle is therefore the next monthly forecast (the 1st, 09:00 IST).

Leaving observe (planned, not implemented). In the current code, `observe` re-proposes one step from the frozen stored file each night, so its records do not accumulate into a learner. SA-043 plans a shadow learner. It learns six factor weights pooled across all stocks (one-engine design), starting from the defaults and learning on the corrected target (SA-012 to SA-015). For each issued decision, it records the default verdict and the shadow verdict from the same factor scores. SA-022 then scores only the decisions where the two disagree. Learning may return only when all of these hold: - at least 100 effective disagreements, over at least 3 months and 3 lenses; - the shadow is right on at least 60% of them; - its wins are consistent month to month and robust to dropping its best lens; - no factor's weight has collapsed to its bound, or below half its default, for 4 weeks.

The pass must hold at two consecutive monthly looks, and the owner still decides. adapt would then resume from the shadow's weights, never from the stored pre-fix weights. A reverse tally in the first month (below 45%) returns learning to observe.

SA-015 requires retry-safe updates and final bounds, SA-016 fixes attribution, and SA-017 records comparable history. SA-020/SA-021 address overlap, calibration and recovery evidence. SA-022 requires prospective comparisons against a frozen policy; SA-023 tests lessons on probation. Future matured market observations remain necessary even after implementation acceptance.

Event ingestion adds company observations; the curator maintains knowledge and questions; the research job attempts answers. These are knowledge tasks, not trades. Discovery paper tracking uses a separate store root with learning writes disabled for that lane; it is not live investor performance.

6. Portfolio: advice and virtual execution

Trace: pipeline → advisor → executor → store → digest.

The pipeline loads active users' portfolios, obtains closes and signals, evaluates held positions, records advice and optionally executes it. The LLM narrates the rule-driven decision; triggers remain available when narration fails.

Research verdict	Portfolio action
STRONG BUY, BUY, NEUTRAL, SELL, STRONG SELL	ADD, HOLD, TRIM, EXIT, SWITCH
Assessment of a stock at analysis time	Action for an existing holding, using entry cost, risk, position size and evidence

A BUY report does not automatically purchase shares. A portfolio HOLD does not mean a flat forecast.

Advisor precedence

The cascade is **EXIT** → **TRIM** → **ADD** → **HOLD**. EXIT includes stop breach, bearish thesis/shock conditions, crisis conditions and an armed trailing stop. TRIM considers profit with declining confidence/reversion. ADD requires bullish, sufficiently healthy conditions and position headroom. EXIT may become SWITCH when an eligible replacement exists. A holding-age rule can soften certain TRIM actions to HOLD; it does not override EXIT. These are product rules, not individualized tax or investment advice.

Decision gate (SA-003). ADD and a SWITCH's buy leg add risk, so in enforce mode they need two things: the holding's close must be from the review session's own bar, and the forecast rows behind the signal must come from an actionable analysis. A blocked ADD becomes HOLD. A blocked SWITCH destination falls back to the next eligible idea, or to a plain EXIT. A shelf idea shelved before SA-003, from an abstained deep dive, or priced from an earlier bar is not a destination. Either way the advice gains the note DATA_GATE and a data_gate record (what was withheld, why, and the source forecast runs). That record is on the user's own advice ledger; the shared gate log never holds a user id. EXIT and TRIM are never blocked: missing data must not trap a position. When they rest on a stale close or an unverified forecast, the advice records that. In record mode the decision is unchanged

and `data_gate` says what enforcement would have withheld. **Example:** a holding's envelope was built on an abstained STRONG BUY and points up. Unenforced, that gives ADD and a virtual buy of 2 shares. Enforced, it gives HOLD with `DATA_GATE`, and no buy.

Execution and persistence

The global gate, user's virtual-autopilot choice and cash-accounting state all matter. Execution reloads the portfolio under a lock, sells before buying, and uses transaction IDs plus a monotonic review-date marker. Future and already-completed dates are rejected/skipped. Cash, sizing and eligibility can prevent a recommended action from becoming a transaction.

Transactions are appended before saving holdings/cash. A crash in between can leave the ledger ahead of the portfolio; deduplication is not an atomic multi-file transaction. Reconciliation handles this gap. Corporate actions affect adjusted quantities and cost. No broker-order execution path was found. In enforce mode the executor also re-checks a SWITCH buy leg's own price: if its close is not from the session's bar, the sell executes and the buy is skipped, and the user gets the existing `switch_buy_skipped` alert.

Where cash goes (current code). The autopilot opens a new position only through a SWITCH, putting one sale's proceeds into a shelf idea. ADD tops up a stock already held, capped at `advisor.max_position_pct` (10%) after the trade. A plain EXIT leaves its proceeds in cash, and nothing reinvests them. The SWITCH buy spends the whole proceeds, with no position or sector cap. **Example (the owner's screen, 27 Sep):** ₹8,09,297 of ₹9,99,611 was cash (81%), with two holdings. ACMESOLAR's `exit_full` alone moved about ₹2.27L to cash, and FEDERALBNK was 13.1% of the portfolio. **Planned:** one sizing rule for every autopilot buy (SA-050). In a normal market, idle cash goes into shelf ideas that pass the checks; cash is held on purpose only in a crisis (SA-049, the owner's decision of 2026-09-27, behind a switch that ships off).

7. Profit/loss and the different marksheets

“Marksheet” covers several distinct reports. Each needs its own question, dates, denominator and exclusions.

Virtual P/L

`portfolio_api.py` exposes holdings, transactions, advice, digest and `/portfolio/performance`.

Measure	Meaning
Holding market value	Adjusted quantity × marked price.
Unrealized holding P/L	Adjusted quantity × (marked price – adjusted average cost).
Realized P/L	Ledger gain/loss on quantities actually sold virtually.
Total equity	Remaining holdings' market value + deployable cash.
Total return	(total equity – capital_in) / capital_in, when required values exist.
As-of date	Date of the valuation, potentially different from page-open time.

The virtual P/L charges no brokerage, STT, stamp duty, exchange fee, DP charge or tax. A real round trip costs about 0.25% of the traded value, and short-term gains are taxed. For ACMESOLAR's +₹37,551, that is about ₹477 in costs and about ₹7,415 in tax (SA-048 plans both). The screen's "Invested" is `capital_in`, the mock money ever put in, not the money in stocks. The screen also has no benchmark (SA-051 plans a Nifty comparison and honest labels).

Example without fees/corporate actions: start with 2,000 and buy ten units at 100. Cash is 1,000. At 110 the holding is 1,100, equity 2,100 and unrealized P/L 100. Sell four at 110: cash becomes 1,440, remaining holding value 660, realized P/L 40 and remaining unrealized P/L 60. Equity stays 2,100.

Current performance reads at most 2,000 transactions and 400 history rows. With no history, a failed price lookup may fall back to adjusted purchase cost. The digest sums priced holdings only, and its `portfolio_value` excludes cash. These scope differences need interpretation; a displayed total is not proof all prices are fresh or all history is included. These virtual results do not establish executable net returns after fees, spread, tax and slippage.

Report families

Report	Present meaning	Limitation / PI work
RL Monitor / analytics	Envelope, feedback, miss and weight records.	Inherits label/timing/history problems; SA-012–SA-017, SA-024.
Weekly scoreboard	28-calendar-day lookback, non-HOLD calls at least five calendar days old, latest supplied closes.	Not fixed matured horizons; current-holding price selection can omit exited stocks. SA-018/SA-019.
Monthly scorecard	Agent/control/simple-baseline lanes and previous-month comparisons.	Completeness, timing and denominators need repair. SA-018.
Learning evidence	Retrospective adapted/base/uniform replay and lesson associations.	Not a prospective controlled experiment of the whole system. SA-020–SA-023.
Nightly advice auditor	Issued advice, alert, shelf, switch and IPO lanes; default 10/30/60 trading-session horizons, and 1/5/21/63/126/252 from listing on the IPO lane.	Separate from weekly heuristics and actual account P/L. IPO rows are excluded from the rendered report.
IPO history report	Subscription groups and issue-price returns at available horizons.	Descriptive associations with sample/feature gaps, not a validated prediction model.

Audit rules judge HOLD/ADD correct when excess return over the benchmark is nonnegative, and TRIM/EXIT/SWITCH correct when negative. The switch-pair lane separately asks whether destination beat origin over the same dates; a tie does not establish benefit. An EXIT can score correctly while the actual virtual trade realizes a loss: these measure different things.

The IPO lane (`grade_ipo_lane`) grades the P3 IPO verdicts under `is_ipo_correct`, a separate rule from the advice vocabulary. Its entry price is the **issue price**, not a market close, and its horizons count trading days from listing with horizon 1 being the listing day itself. Only one row per issue can carry a True/False: the listing-day horizon, and only when the verdict was taken after the book closed and its lean asserted a direction. Every other horizon is recorded with `correct` unset, so it enters no hit-rate. These rows are written to the same store and excluded from the rendered audit report: the P3 model remains dark until its visibility gate is met, and a displayed hit-rate would be that surface. No production IPO row has been graded yet.

Example: stock +5%, benchmark +8% means excess **-3 percentage points**. HOLD is incorrect under that relative-return rule despite a positive stock return. Every marksheet should identify its rule, horizon, matured sample count, missing data and excluded cases.

8. Discovery and IPO functionality

Discovery

run_discovery_cycle connects EOD sync, bulk/block activity, optional recent-IPO candidates, screen, deep dives, conviction shelf and paper reviews. Failures can degrade individual stages. The Saturday job and manual discovery endpoint use this entry point. A shelf candidate, watchlist promotion and virtual purchase are separate steps.

IPO layers

Layer	Current source and output
Calendar, offer and bids	ipo.py, ipo_offer.py, ipo_bids.py: normalize and enrich issues. Failed refresh preserves previous cache with degraded/stale information.
Captured snapshots	signals.py: observed refresh facts with hour/content deduplication. velocity.py: demand changes derived on read.
Historical evidence	history.py, outcomes.py, report.py: facts, realized curves and subscription-bucket summaries.
P3 model and deep dive (dark)	research.py, extract.py: browsed, corroborated Substance facts with source URLs. hype.py, substance.py, verdict.py: deterministic indices and the §3 verdict grid over captured and browsed facts. deep_dive.py: the 19:00 ipo_deep_dive sweep (T-1 research run, post-close re-read from cache) that writes to the append-only store in verdicts.py. narrate.py: a sourced research note stored on that row, written by the bulk model from the SAME structured findings the indices read — the verdict is never passed to it, every number in the prose must appear in the findings, an advice/verdict vocabulary rejects it, and a rejected or failed note falls back to a deterministic template. Writes only: no verdict, index or quadrant reaches any surface until the ipo_verdicts_visible_gate milestone is judged on forward rows.
Forward grading of the P3 verdict (dark)	listing.py resolves the listing date and issue price from the P1 spine, then the NSE cache; grade_ipo_lane grades the newest stored verdict per issue against the tape at 1/5/21/63/126/252 trading days from listing, entry price = issue price. Only the listing-day row of a post-close verdict with a directional lean can be scored; the rest carry the return and no claim. Rows land in the existing per-user audit store and are excluded from the rendered audit report . No production row exists yet: the job reaches production only on deploy, and grading also waits on the issue reaching the P1 spine, which scripts/ipo_backfill.py still rebuilds manually.
Recent-listing ranking	ipo_tracker.py: listing evidence, delivery trend, bulk accumulation and optional subscription score discovery candidates.
User surfaces	IPO-watch in briefs, weekly/discovery context and shelf. The brief's demand lean (_ipo_lean) judges subscription against size-tiered bands from delivery.brief_ipo_size_tiers; an issue whose size was not read uses the scale-free scalar thresholds, never the largest tier. The lean is a labelled heuristic, not the P3 verdict. Inspected routes do not provide a dedicated /ipo/predict API or complete standalone IPO prediction page.

History uses **1, 5, 21, 63, 126 and 252 trading sessions**, measured from **issue price**. Unmatured horizons are absent, not zero. Benchmark excess subtracts the listing-to-horizon benchmark return; that window does not capture the stock's issue-to-listing move in the same way. Do not interpret the result as a fully matched investable strategy. Missing subscription is excluded, and report bucket means below a sample count of ten are suppressed.

The recent-listing tracker is a separate heuristic. It needs at least five EQ sessions and configured price/liquidity guards. Component weights are 40% listing evidence, 20% delivery trend, 15% bulk accumulation and 25% subscription, renormalized over available components. It can fall back to first cached close when issue price is absent; that differs from historical-outcome calculation.

GMP fetching exists behind a separate disabled-by-default gate. Missing GMP does not mean zero. Lock-in alert dates use fixed 30/90/180-calendar-day offsets from listing; these are software heuristics, not verification of an individual issue's contractual dates.

The historical **Prospect** design planned further modeling after P0/P1/P2 collection. Current code has collection, history, captured signals, heuristic candidate ranking and, since PI Prospect Sprints 2–4, the dark P3 model, deep-dive job, narrator and forward-grading lane in the table above. None of it reaches a user, and no production verdict row has been graded. A validated IPO application/allotment or listing-gain predictor is therefore still not established, and is not silently added to the September remediation scope.

9. Scheduled jobs and event hooks

Source: scheduler.py. These are in-process APScheduler jobs, not separate OS cron jobs. There are **24 possible job IDs**, including three IPO jobs. Registration depends on gates and valid cron configuration; registration alone is not execution. All times are **Asia/Kolkata (IST)** defaults.

Job ID	Schedule	Work / condition
prompt_daily_deploy	Daily 00:00	Prompt publishing; handler/configuration can write externally.
ops_watchdog	Daily 06:30	Milestones/checks/configured prep; watchdog gate.
macro_daily_news	Mon–Fri 07:30	Policy/RBI feed; macro-news gate.
ipo_refresh_am	Daily 08:00	Calendar/bids/snapshots; IPO gate.
preopen_shock_check	Mon–Fri 08:45	Shock/reforecast; feature/calendar checks in handler.
morning_brief	Mon–Fri 08:50	Brief assembly/delivery; delivery gate.
macro_market_news	Mon–Fri 09:00, 12:00, 15:00	Intraday macro feed; macro-news gate.
r1_daily_review	Mon–Fri 16:30	Previous trading session; configured feedback cron.
ipo_refresh_pm	Daily 17:45	Evening IPO update before Sunday weekly review; IPO gate.
ipo_deep_dive	Daily 19:00	T–1 IPO research + post-close re-read into the dark verdict store; IPO gate.

Job ID	Schedule	Work / condition
bhavcopy_daily_sync	Mon–Fri 19:00	Official EOD cache; discovery gate.
atlas_universe_recompute	Daily 23:00	Demand/cadence; no-op if Atlas disabled.
atlas_cost_rollup	Daily 23:15	Cost aggregation; Atlas gate in handler.
atlas_retention	Daily 23:20	Retention; Atlas gate in handler.
data_backup_nightly	Daily 23:30	Archive with manifest, restore drill, local rotation, encrypted off-site copy when configured, email copy; the status feeds the watchdog (SA-007).
audit_nightly	Daily 23:45	Matured outcome grading/breach reports; audit gate.
ledger_cleanup_weekly	Monday 03:30	Stale-lesson cleanup.
event_ingest_weekly	Saturday 10:00	Dossier events; event-ingest gate.
research_loop_weekly	Saturday 11:00	Open company questions; research-loop gate.
discovery_weekly	Saturday 12:30	Screen/deep dives/shelf/paper; discovery gate.
weekly_review	Sunday 18:00	Portfolio summary/index watch; delivery gate.
scorecard_monthly	Calendar first 02:00	Previous-month scorecard/evidence; scorecard gate.
r1_monthly_forecast	Calendar first 09:00	Forecast envelopes.
r1_calendar_update	December 31 23:00	Holiday calendar refresh.

Startup self-heal, the post-review portfolio/digest hook and the outbox drainer are **not additional cron jobs**. An 08:50 brief is not guaranteed to wait for an unfinished 08:45 shock check. Close clock times are not dependencies.

Daily-review outcomes (SA-004: accepted 2026-09-27, committed as 241c393, production verification pending). Before SA-004 the job counted every review that did not raise as output. On 2026-09-08 it recorded produced=20, expected=20 and pipeline_ok=true, while WELCORP returned no_envelope and 19 feedback rows existed. Now review_outcomes.py gives each enabled ticker exactly one outcome:

- completed: the review wrote its feedback entry;
- degraded: it wrote its feedback entry on an input the data gate would have stopped (SA-003 record mode lists them). Until the 1 Oct envelopes, most reviews grade a row issued before the gate existed, and those read degraded;
- data_gated: the gate stopped it before any write (enforce mode);

- `skipped`: no envelope, no forecast row for the session, or no close;
- `failed`: it raised, returned a malformed result (an unknown status, or a result for another ticker or date), or was still running when the harvest budget ran out.

Only `completed` and `degraded` are output (produced), so a skip, a gate stop or a failure fires the zero- or partial-output alert, which now names the tickers and why. **Example:** 20 tickers, and WELCORP has no envelope. The record reads 19 of 20, `status`: `partial` and `missing`: `{WELCORP: "skipped: no_envelope"}`. The alert says "completed 19/20 — missing: skipped no_envelope: WELCORP".

- Required work (the distinct enabled tickers) and attempted work are counted separately. Disabled tickers are listed as `excluded`, and a duplicate entry is reviewed once and listed under `duplicates`.
- The day after a weekday NSE holiday reviews the same session again: Fri 2 Oct and Mon 5 Oct both review Thu 1 Oct. The second run merges with the first, so attempts add up and completion does not. A ticker whose earlier run wrote its feedback stays output even if the rerun fails.
- `status` is `ok`, `partial`, `failed` or `empty` (every ticker disabled). `pipeline_ok` is true only when the portfolio pipeline returned a well-formed `completed` and no required review failed; `pipeline_status` says what the pipeline itself did.
- The record in `data/scheduler_job_outcomes.json` keeps the legacy fields and adds `by_ticker`, with each ticker's outcome, reason and attempts.

Still open. The pipeline's own `completed` does not count holdings whose advisor call failed inside it (routed to SA-036). The learning side of a rerun is F10's (SA-015). Nothing in the watchdog reads this record yet (SA-034). HTTP 202 means accepted for background work, not finished. SA-011 covers the operational backlog, and SA-029 covers readiness and ownership recovery.

10. Delivery, interface, identity and operations

Briefs, weekly reviews and digests persist independently of transport. Channels send email and web push. Email has two transports: SMTP (STARTTLS) and the HTTPS Resend API. `EMAIL_TRANSPORT` in settings defaults to `auto`, which uses Resend only when `RESEND_API_KEY` is set and SMTP otherwise; `resend` or `smtp` forces one. The repository's `delivery.email_enabled` is `false`; the `DELIVERY_EMAIL_ENABLED` environment variable overrides it.

When its Atlas path is active, `outbox.py` holds one row per message and channel, and a drainer in the singleton owner sends each row. Stored, queued, transport-accepted, received and read are separate states; the code observes only the first three. A row's stored status `delivered` means the transport accepted it.

Delivery contract (SA-006: accepted by its fresh re-review 2026-09-28, after two review fixes made the same day; committed as 15dcda1; not yet deployed). Every transport now returns one result: whether it accepted the message, why not, whether a retry can help, and any wait the provider asked for. The drainer acts on it:

- **Accepted is not received.** SMTP 250, a Resend 2xx and a push service's 201 are acceptance. The row records the transport in `accepted_by`, with Resend's message id, which the owner can look up in the Resend dashboard. No receipt, bounce or display is observed.
- **Transient failures retry.** A network error before anything was sent (for push, only a connection that was never made: refused, a failed DNS lookup or a connect timeout), an SMTP 4xx reply, or HTTP 429 or 5xx:

the row retries after 1 and then 5 minutes (the configured backoff), or later if the provider's Retry-After asks for more (capped at 6 hours). After 3 attempts it is dead-lettered as "retries exhausted".

- **Permanent failures stop at once.** A rejected login, an SMTP 5xx reply, Resend 400/401/403/422, a disabled or unconfigured channel, no push subscription, or no address for the account: one attempt, then a dead letter whose reason names what to check. For example, a Gmail 535 reads "check SMTP_USER and SMTP_PASSWORD (Gmail needs an app password)".
- **At most once.** When the outcome is unknown, the row is dead-lettered as "unknown outcome" and never re-sent: the SMTP connection dropped inside the send; a push failed in any way other than an HTTP status or a connection that was never made (a read timeout, or the connection dropping after the request was sent, when the push service may already have stored it); or the process stopped while the row was sending. SMTP and web push cannot deduplicate a repeat. So if one of an account's phones gives an unknown outcome, the row stops for all of them, and a phone whose push service answered 5xx loses that notification, visibly, as a dead letter. Email over Resend carries a per-row Idempotency-Key, which Resend keeps for 24 hours, so its timed-out requests do retry without a second email. A sending row counts as abandoned after 60 minutes; a younger one may belong to the old container during a deploy.
- **Recipient isolation.** `resolve_recipient()` returns the owning account's address from `users.db`. `DELIVERY_EMAIL_TO` is used only on the single-user path: no user id, or the default portfolio id. For any other account, a failed lookup retries and a missing address dead-letters; neither reaches the owner. The inline path (Atlas off, or the outbox unreachable) follows the same rule; before SA-006 it mailed every user's message to `DELIVERY_EMAIL_TO`.
- **Redaction.** Reasons and log lines mask email addresses, URL paths (a push endpoint is a bearer capability) and the configured secrets.
- **Push TTL.** Pushes carry a 12-hour TTL (`delivery.push_ttl_seconds`). pywebpush's default of 0 lets a push service drop a notification for a phone that is offline when it arrives. This comes from reading the library; its effect on what users saw was not measured.
- **Visibility.** `GET /delivery/outbox` (owner session or machine key) returns counts per channel and status, with `delivered` shown as `accepted`; the newest dead letters and retrying rows with their reasons; and the last acceptance per channel. It states that user receipt is not observable, and it carries no payload, address or user id: a dead letter for an account with no address reads "no email on file for this account", and its row id identifies it.
- **History.** Nightly retention deletes accepted rows after 30 days. Dead letters stay for 180 days; after 30 days only their payload is cleared.

The monthly Learning Evidence email, the watchdog heartbeat and the backup email are sent directly, not through the outbox, so they get no retry and no dead letter. A failure is now logged with its reason. The backup email is no longer the only off-site copy (see "Backups and recovery" below), and SA-042 adds a witness outside the app.

Dated production observation. September 10 recorded email failures and no confirmed app-created off-site backup copy. The email cause was identified on 2026-09-21: Railway disables outbound SMTP on its Hobby plan, and production had logged `[Errno 101] Network is unreachable` on every send since 2026-07-16. After a plan upgrade and redeploy, one triggered brief reached the inbox that day. A read-only probe on 2026-09-23 counted, since that redeploy, 8 email rows accepted and 0 dead, and 11 push rows accepted. Before 21 Sep, 77 email rows were dead and 77 push rows accepted. Those counts are provider acceptance, not a measured receipt rate. Whether production sets `RESEND_API_KEY` was not inspected. The SA-006 code above has not been deployed.

Remaining limits:

- Resend's default shared sender (`RESEND_FROM`) delivers only to the Resend account owner's address. Per-account email over Resend needs a verified-domain sender, which is an owner configuration step, not

code.

- When several push subscriptions exist, acceptance by one counts as accepted; the others are not retried, so they cannot receive a duplicate.
- Resend deduplicates a key for 24 hours. Retries end well inside that window (two waits, each capped at 6 hours); a policy with longer waits would lose that protection.

Backups and recovery (SA-007: accepted by its fresh review on 2026-09-29; committed as 165d152; deployed on 2026-09-29 as 26b442f4. Change 1, a weekly reminder while no target is configured, was accepted by its fresh review the same day and is not yet committed). Before SA-007 the nightly backup zipped data/ onto the volume it protects and emailed the zip. Only `telemetry.db` and `scores.db` went through SQLite's backup API. `users.db`, `atlas.db` and `chat_sessions.db` run in WAL mode and were copied as raw files, so rows still in their `-wal` files were lost: restored alone, a raw copy of a fresh WAL database has no tables at all. Measured 2026-09-23: the job built an 8.98 MB zip, the email failed, and the only copy stayed on the volume.

Each night at 23:30 the job now runs these steps:

1. **Build.** Every non-cache file under `data/` goes in. A file is treated as SQLite by its header, not its name, and is copied through the backup API; `-wal`, `-shm` and `-journal` files are never copied raw. Credential-shaped files (`.env`, `*.pem`, `*.key`, `*secret*` and similar) are left out and logged. Other files are read until one read sees no size or modification change. `MANIFEST.json` inside the zip records each file's size and SHA-256. For each database it records `integrity_check`, a schema digest and every table's row count, and for each portfolio ledger its row count. A sidecar `<archive>.manifest.json` records the zip's own size and SHA-256. The zip is built under a `.part` name, so an interrupted run leaves no archive.
2. **Drill.** restore restores the archive into a fresh temporary directory and recomputes everything the manifest claims. Errors mean the copy is not faithful: a missing manifest, a checksum mismatch, a corrupt zip, an unexpected or unsafe member, a failed `integrity_check`, or a schema or row-count difference. Warnings mean the copy is faithful but the data is suspect. The drill also checks ledger continuity. For example, Monday's archive holds `transactions.jsonl` at 1,200 bytes, so Tuesday's must start with those same 1,200 bytes, because the four portfolio ledgers are append-only. If a script rewrote row 3 overnight, Tuesday's drill warns "was rewritten"; a ledger that shrank or disappeared warns the same way. Older off-site copies still hold the earlier history.
3. **Rotate** the local copies to the newest 7, each with its sidecar.
4. **Off-site.** Only an archive that passed its drill leaves the volume. offsite encrypts it with AES-256-GCM under `BACKUP_ENCRYPTION_KEY` and uploads it to `BACKUP_OFFSITE_TARGET`. That is `s3` (any S3-compatible bucket, signed with AWS Signature V4 using only the standard library and `requests`) or `dir` (a separately mounted directory, refused when it overlaps `data/`). The ciphertext goes first and the manifest last; each must read back at its uploaded size. Only then is the copy "confirmed". An upload interrupted halfway is never confirmed, and its leftover ciphertext is deleted once a newer copy is confirmed. Retention keeps the newest `BACKUP_OFFSITE_KEEP` confirmed copies (30 by default; 0 or less keeps all). It never touches objects not named like a backup, and it deletes a copy's manifest before its ciphertext.
5. **Email** the plaintext zip as before, when it is under 20 MB. This is a direct send; it does not count as off-site.
6. **Record** `data/backups/backup_status.json`. A failed drill then raises, so the scheduler's job-error alert fires.

Visibility. The watchdog invariant `backup_recoverable` reads that status every morning. It warns when the job has not run for 36 hours, when the drill failed, when last night's copy was not confirmed (the reason is scrubbed of secrets), and when the drill flagged a ledger. Until the owner configures a target it says "No off-site copy has ever been confirmed", **once a week**. On 2026-09-29 the owner deferred the bucket to the end of the PI and chose a weekly reminder over a daily one (SA-007 change 1, accepted by its fresh review and not yet committed; until it is

deployed, production still warns daily). The check asks for that pace itself (`repeat_days=7` on its result), so everything else still warns daily: a failed drill, a stopped job, a flagged ledger even with no target, and a half-done setup such as a target without a key. For example, a reminder on Thursday is followed by silence until the next Thursday. If Friday's drill fails, Saturday's watchdog warns anyway. One gap remains (review F1, routed to SA-034): a night on which the job never runs, for example because the container restarted at 23:30, is not reported. At 06:30 the newest status is then 31 hours old, under the 36-hour limit, so the check still reads the previous night's copy as confirmed. Two missed nights in a row are reported.

A ledger rewrite is reported on one morning only, because the next night compares against the rewritten archive. The earlier history then survives only in the older off-site copies, for `BACKUP_OFFSITE_KEEP` nights.

Owner configuration (not provisioned by code). Create a private bucket and an access key limited to it. The archive is about 9 MB, and several S3-compatible providers have a free tier at that size. Set these Railway variables:

- `BACKUP_OFFSITE_TARGET=s3`;
- `BACKUP_S3_ENDPOINT` (a bare `https://` URL), `BACKUP_S3_BUCKET`, `BACKUP_S3_REGION` (auto by default) and optionally `BACKUP_S3_PREFIX` (stockagent/);
- `BACKUP_S3_ACCESS_KEY_ID` and `BACKUP_S3_SECRET_ACCESS_KEY`;
- `BACKUP_ENCRYPTION_KEY`, from `python -c "import os,base64;print(base64.b64encode(os.urandom(32)).decode())"`.

Keep a second copy of the key outside Railway, for example in a password manager: without it no off-site copy can ever be read. Setting a Railway variable redeploys the service.

Restore runbook. On a machine with the same variables set, run `python -m services.data.restore fetch --dest <empty directory>` from the repository root. It downloads the newest confirmed copy (or a named one), checks the ciphertext against its manifest and decrypts it; a wrong key or one flipped byte stops it. It then drills the result into `<dir>/restored`. Exit codes: 0 is clean, 2 is faithful with warnings, 1 is failed. `python -m services.data.restore drill <archive>` checks a local archive the same way. Neither ever writes into `data/`: a destination that overlaps it, or is not empty, is refused. Putting restored files back into a stopped deployment is a deliberate human step.

Limits.

- Each file is a point-in-time copy, but the archive is not one snapshot across files. The job runs at 23:30, when no trading job writes.
- One malformed SQLite file stops the whole archive (review F2, routed to SA-036). The job raises, so the job-error alert fires each night. No new archive, off-site copy or status is written until the database is repaired, so the ledgers are not backed up in the meantime. Earlier archives and off-site copies are kept.
- The email copy is still an unencrypted direct send.
- Credentials live in Railway variables and are not backed up.
- A deleted account's data survives in off-site copies until retention removes them (30 nightly copies by default).
- Off-site recovery counts as verified only after a recorded `fetch` drill against the real bucket. None has been run: no bucket exists yet.

The frontend uses React JSX, runtime browser transformation and PWA assets. Chat uses a streaming tool loop with potentially paid provider calls. Prompt editing can feed scheduled publishing. These are not inert testing screens. Bearer sessions, roles and machine-key authentication are implemented; portfolio identity comes from

the authenticated user. Some intelligence read routes remain public. The RL client has demo/fallback paths on API failure, so populated charts alone do not prove live data. SA-024/SA-032 address evidence and public/demo policy.

Chat rendering (SA-001: accepted by its fresh review on 2026-09-26; committed as 8413b59; production verification pending). Chat replies are untrusted text: the model quotes news snippets and tool output. Before SA-001, sphere.jsx passed them through marked straight into dangerouslySetInnerHTML, so a quoted `<img src=x onerror=...>` ran its handler in the browser (audit F01), next to the stored bearer token. Now renderMd delegates to a new plain script, chat-markdown.js. index.html loads it after marked 12.0.2 and DOMPurify 3.4.16, both exact versions with SRI hashes. There are two layers:

- marked shows any raw HTML in a reply as visible text; the chat prompt asks for Markdown only.
- DOMPurify then keeps only Markdown's own tags. Links must be http(s) and open in a new tab with `noopener noreferrer`. Images, styles, classes and event attributes are dropped.

For example, a reply quoting `<img src=x onerror=...>` now shows that string as text; bold, lists, code fences, tables and https links still render. If either library fails to load, the reply is shown as escaped text. The raw parser is taken off window, so a pre-fix sphere.jsx still cached by the service worker also falls back to text; the worker's cache version moves to v8. That bubble is the client's only HTML sink, and a unit test keeps it so. `npm run test:frontend` drives the real ChatOverlay in Chromium with hostile fixtures and no network.

Durable logs connect run IDs, tickers, warnings, LLM usage and health. Bundle call counters are not complete nested-provider/fallback cost accounting; SA-025 addresses this. The watchdog reads milestones.yaml, runs checks, persists state and may perform configured preparation. A sent milestone notice is not milestone completion. Local backup files and healthy HTTP responses do not prove recovery or successful jobs. The nightly drill proves that an archive restores; only a recorded off-site fetch proves recovery without the volume.

11. Changes already present and planned redesign

Topic	Current implementation	Still planned / unverified
Sector routing	Shared graph selection via registry.	Complete store lineage, and sector lenses resolved from NSE's industry field (one-engine design; SA-026).
Analysis	Unified scoring plus surviving legacy fallback.	Actual call accounting. A factor engine (computed factors, one text reader, code decides, LLM explains) proven in shadow, then the graphs and fallback retired (SA-044–SA-047, SA-027).
Data health	Durable health/run records. Producer-typed section status and usable-data health (SA-002, accepted 2026-09-26, committed as 8413b59). A decision gate on essential data across research, learning and portfolio (SA-003, accepted 2026-09-27, committed as 167f08b), shipped recording only.	A measured record period, then the owner's decision to enforce; instrument lifecycles (SA-008).

Topic	Current implementation	Still planned / unverified
Verdict binding	Deterministic category enabled in YAML, raw model verdict logged.	Correct issue-time grading and final adaptive constraints.
Portfolio	Per-user advice/execution, stops, switches and ledgers.	Stronger upstream evidence and report reconciliation; costs and tax in paper P&L (SA-048); idle cash put to work in normal markets (SA-049); one sizing rule for every autopilot buy (SA-050); the portfolio against the Nifty, with honest labels (SA-051).
IPO	Calendar, history, snapshots, recent-listing screening, size-tiered brief lean, and the dark P3 model, deep dive, narrator and forward-grading lane (section 8).	Forward evidence for P3 and its ipo_verdicts_visible_gate; no verdict reaches a user; outside default September scope.
Operations	TCP singleton, outcomes, watchdog, outbox with last_error, per-account recipients, SMTP/Resend transports and backup code. Truthful daily-review outcome counts (SA-004, accepted 2026-09-27, committed as 241c393). SA-006 (accepted 2026-09-28, committed as 15dcda1, not yet deployed): transient-versus-permanent retry, at-most-once sends, no owner fallback for accounts, and the GET /delivery/outbox dead-letter view. SA-007 (accepted 2026-09-29, committed as 165d152, deployed 2026-09-29 as 26b442f4): archive manifests, a nightly restore drill, an encrypted off-site adapter and the backup_recoverable watchdog check.	Durable outcomes for every job (SA-036), an off-site bucket configured by the owner and a recorded off-site restore (SA-007), measured receipt (only acceptance is observed), SA-006's deployment, and readiness. The observability design adds planned job-run and source-health ledgers, post-job checks, a read-only status fetcher and an outside witness (SA-034–SA-036, SA-040–SA-042).
Frontend	JSX/PWA with live adapters and some fallback/demo paths.	Sanitization, honest unavailable states and optional build cleanup.

Historical specifications retain what was intended at the time. The September audit supersedes unsupported August claims about correct learning, automatic security succession or unused fallback cost. Legacy code should only be retired with measured replacement coverage.

12. PI changes included now

The table below includes the planned destination now. **SA-039 is accepted (production observe since 2026-09-26). SA-001 and SA-002 (after a rework) are accepted and committed as 8413b59; their production verification is pending. SA-003 is accepted by its fresh review (2026-09-27) and committed as 167f08b; its production verification is pending. SA-004 is accepted by its fresh review (2026-09-27) and committed as 241c393; its production verification is pending. SA-005 is accepted by its fresh review (2026-09-28) and committed as 48143ed; its CI workflow has not run yet. SA-006 is accepted by its fresh re-review (2026-09-28). Its first review asked for two fixes the same day: a push could reach a phone twice, and the owner report could show a user id. Both are made; it is committed as 15dcda1 and not yet deployed. SA-007 is accepted by its fresh review (2026-09-29), with two low follow-ups routed to SA-034 and SA-036; it is committed as 165d152 and deployed as 26b442f4 (2026-09-29). Its change 1, a weekly missing-bucket**

reminder, is accepted and not yet committed. Every other SA story is todo. Accepted state/dependencies are in STATE.json. DOC-001 is this user-requested documentation refresh; it does not close SA-031 or any upstream remediation.

Story	Planned change	KT sections	Dependencies
SA-001	Sanitize every chat Markdown rendering boundary	10 Interface	None
SA-002	Make data-health records describe usable evidence	4 Research	None
SA-003	Gate recommendations and learning on essential data	4 Research; 5 Learning; 6 Portfolio	SA-002
SA-004	Count daily-review outcomes truthfully	9 Jobs	None
SA-005	Establish a clean, isolated test and CI baseline	13 Validation	None
SA-006	Repair delivery transport and expose dead letters	10 Delivery	None
SA-007	Make backups independently recoverable	3 Storage; 10 Recovery	None
SA-008	Quarantine unresolved securities with explicit lifecycle records	4 Identity; 8 Discovery	SA-003
SA-009	Inventory and reconcile prediction-store ownership	3 Storage	None
SA-010	Pass sector benchmarks through technical-data calls	4 Research	None
SA-011	Turn durable errors into a deterministic operations backlog	9 Jobs; 10 Operations	SA-002, SA-004
SA-012	Specify and test the learning target before changing it	5 Learning; 7 Marksheets	SA-005
SA-013	Persist immutable, genuinely prospective forecast identities	3 Storage; 5 Learning	SA-012
SA-014	Grade frozen decisions and issue controls before their outcomes	5 Learning; 7 Marksheets	SA-003, SA-013
SA-015	Make adaptive updates idempotent and preserve final weight bounds	5 Learning	SA-014
SA-016	Replace blame-based shared credit with identifiable outcomes	5 Learning	SA-015
SA-017	Attach historical lineage and quarantine incomparable feedback	3 Storage; 5 Learning	SA-009, SA-013
SA-018	Build complete, matched monthly evaluation cohorts	7 Marksheets	SA-014, SA-017

Story	Planned change	KT sections	Dependencies
SA-019	Evaluate suggestions across weekly and fortnightly cohorts	7 Marksheets	SA-018
SA-020	Compute effective samples from trading-session overlap	5 Learning; 7 Marksheets	SA-013
SA-021	Validate calibration and weight recovery with minimum evidence	5 Learning; 7 Marksheets	SA-016, SA-018, SA-020
SA-022	Run prospective adapted and frozen-policy experiments	5 Learning; 7 Marksheets	SA-016, SA-018, SA-020, SA-043, SA-047
SA-023	Put learned lessons on measurable probation	5 Learning	SA-022
SA-024	Show verifiable learning evidence and honest unavailable states	7 Marksheets; 10 Interface	SA-019, SA-020, SA-022
SA-025	Count actual provider calls and fallback cost	4 Research; 10 Cost	SA-011
SA-026	Consolidate sector definitions into sector lenses	2 Modules; 4 Research	SA-003, SA-009, SA-010
SA-027	Retire redundant fallback only when measured replacements work	4 Research; 11 Changes	SA-025, SA-026, SA-047
SA-028 (stretch)	Repair packaging and compile the browser client	3 Runtime; 10 Interface	SA-001, SA-005
SA-029	Verify readiness and recover background ownership	3 Runtime; 9 Jobs	SA-004, SA-007
SA-030 (stretch)	Reconcile or retire stale Atlas projections	3 Storage	SA-009, SA-013
SA-031	Verify final architecture and operator-documentation consistency	All: final consistency check	SA-024, SA-027, SA-029
SA-032 (stretch)	Define and enforce the public-read and demo-data policy	10 Interface	SA-001, SA-024
SA-033	Lock reproducible runtime dependencies	3 Runtime; 13 Validation	None
SA-034	Detect silent degradation in the watchdog	9 Jobs; 10 Operations	SA-036, SA-040
SA-035	Surface LLM and search provider credit exhaustion	10 Operations; 10 Cost	None
SA-036	Record durable outcomes for every critical job	9 Jobs	None
SA-037	Keep deploys from dropping scheduled jobs	3 Runtime; 9 Jobs	SA-036
SA-038	Judge and retire lapsed production-verification milestones	9 Jobs; 10 Operations	None
SA-039	Contain adaptive learning: observe-only, live weights back to defaults	5 Learning	None

Story	Planned change	KT sections	Dependencies
SA-040	Record per-source fetch health and detect new error types	9 Jobs; 10 Operations	None
SA-041	Provide a read-only production status fetcher	10 Operations; 13 Validation	SA-036, SA-040
SA-042	Add an outside witness for missed jobs (Healthchecks.io → Telegram)	9 Jobs; 10 Operations	SA-036
SA-043	Run a shadow learner and record paired decisions in observe mode	5 Learning	SA-015, SA-039, SA-045
SA-044	Compare valuations only with same-industry peers	4 Research	None
SA-045	Compute the five universal factors deterministically	4 Research	SA-026
SA-046	Read text into dated events and a Catalyst factor	4 Research	SA-026
SA-047	Decide with the factor engine; switch when it is not worse	4 Research; 5 Learning	SA-045, SA-046, SA-014, SA-020
SA-048	Charge realistic trading costs and tax in paper P&L	6 Portfolio; 7 Marksheets	None
SA-049	Put idle cash to work in normal markets	6 Portfolio; 8 Discovery	SA-050, SA-051
SA-050	Apply one sizing rule to every autopilot buy	6 Portfolio	None
SA-051	Show the portfolio against the Nifty, with honest labels	7 Marksheets; 10 Interface	None

Maintain documentation with each story

Each implementation updates the affected current-code section, identifies which target behavior has landed, adjusts relevant human test cases and regenerates the PDF if this Markdown changed. Evidence records the tested revision and the difference between implemented and production-verified. SA-031 is the final cross-document consistency check, not a reason to postpone normal documentation until Sprint 6.

13. Validation and KT sequence

The receipt records exact tests, source/path checks, PDF checks and limitations; the review records what was verified independently and what was sent back. Local validation uses Python 3.13 on Windows; production Docker uses Python 3.11 on Linux. Focused checks do not imply full-suite or platform-parity acceptance.

Maintainer commands, with required local dependencies installed:

```
python scripts/docs/run_kt_checks.py
python scripts/docs/build_kt_pdf.py
python scripts/docs/check_kt_docs.py
```

The test runner copies tracked source to ignored `analysis_data/`, excludes `.env/runtime` stores, disables `dotenv`, and blocks external transports and test subprocesses. It exercises a fixed selection of existing tests, not all independent financial invariants. The PDF builder needs Python `markdown2` and local Playwright/Chromium; it does not start the application.

The suite's hermetic boundary (SA-005, committed as 48143ed)

Every `pytest` run now passes through `tests/hermetic.py`, which `tests/conftest.py` imports before any application code. The same four rules hold on a laptop with a `.env` and on a fresh CI checkout:

Rule	What it does	Example
No <code>.env</code> , no shell settings	<code>dotenv</code> is off, and every variable the application reads is removed, so settings come from <code>config.yaml</code> and the code fallbacks.	A developer's <code>.env</code> says <code>RL_LEARNING_MODE=observe</code> ; the suite still runs the shipped <code>adapt</code> .
No outbound network	Non-loopback sockets, DNS and <code>curl_cffi</code> (yfinance's transport, which bypasses Python sockets) raise <code>OutboundNetworkBlocked</code> . A test that triggers one fails, even if the application swallowed the error.	An unmocked thesis review used to call <code>OpenRouter</code> ; the test now fails, naming <code>thesis_reviewer.py</code> and <code>openrouter.ai:443</code> .
An empty working directory	The application keeps runtime state under <code>data/</code> and <code>outputs/</code> relative to the working directory, as under <code>/app</code> on Railway. Each test runs in its own empty temporary directory, seeded only with the three tracked files the application reads by relative path.	A test that toggles a managed ticker changes its sandbox copy, not the checkout's <code>data/managed_tickers.json</code> .
The checkout is off limits	An audit hook refuses any access to the checkout's <code>data/</code> , <code>logs/</code> and <code>outputs/</code> , and any write elsewhere in it.	A test reading local MARUTI predictions fails; the discovery test now builds its own predictions tree.

Measured before the change (2026-09-27, the whole `tests/` tree): the old socket guard saw 74 blocked attempts, but yfinance's `curl_cffi` requests were invisible to it. With the new boundary, the first run found 286 Yahoo requests, 178 OpenRouter attempts and 124 NSE session attempts. Before it, the tests wrote 149 paths in the checkout (35 targets once temp files are grouped) and read 24 untracked runtime files. Most network traffic came from one test that booted the full app: its startup ran the RL self-heal thread, which rebuilds every missing envelope in the background. Those tests now boot the app as an API-only worker. The receipt lists every fix.

The boundary does not reach child processes (the eval CLI smoke test and the browser suite); they inherit only the cleaned environment. It sees Python's own socket, DNS and file calls, and `curl_cffi`. It does not refuse a directory listing or existence check, a native library's own file access (pyarrow, or SQLite given a `file: URI`), a Windows short-name spelling of the checkout, or a numeric-IP connection on the Windows `asyncio` loop. The application builds one absolute checkout path, the startup calendar check, and the test fixture stubs it (SA-029). A Windows-only retry in `core/utils/atomic_io.py` absorbs the transient `PermissionError` that made three tests flaky on Windows; Linux renames never retry.

The CI workflow, `.github/workflows/ci.yml`, runs three jobs on Linux with Python 3.11 and no secrets: the whole `tests/` tree; the broad-exception guard (`scripts/ci/check_broad_except.py`) and `check_kt_docs.py`; and the SA-001 browser suite in Chromium. The guard fails when new code catches every exception without logging, re-raising or

using it. The 154 handlers that already did so are grandfathered in a burn-down list; they were not reviewed one by one. The list counts handlers per function, so a change that fixes one grandfathered handler and adds another in the same function passes. Test-only tools are pinned in requirements-test.txt. The runtime packages still come from the unpinned requirements.txt until SA-033 adds a lock.

KT session	Walkthrough
Product/data	Sections 1–4: a symbol, source dates, dimensions and research verdict.
Predictions/learning	Section 5: envelope row to feedback, weights and lessons; explain the grading gaps.
Portfolio/marksh eets	Sections 6–7: advice to transaction, the P/L example and different evaluation rules.
IPO/discovery	Section 8: calendar to captured facts, history, post-listing candidates, and the dark P3 path from deep dive to a stored, narrated and graded verdict.
Operations/road map	Sections 9–12: scheduled work to persisted output, outbox states, last_error and the dead-letter view, the dated email observation, and remaining PI changes.

Practical, non-code-intensive duties are in the separate Team Human Testing Guide. Testers should be able to explain outputs and compare evidence without reading prompts or reviewing code.